Biologically-Inspired Artificial Intelligence

Genetic Algorithm for the Travelling Salesman Problem and CartPole-v1 Control

BIAI Course Project Report

Piotr Krupiński

Jeremi Szczotka

Politechnika Śląska, Faculty of Automatic Control, Electronics and Computer Science

June 2026

Abstract

This report describes the design and implementation of a genetic algorithm (GA) framework applied to two distinct optimisation problems: the Travelling Salesman Problem (TSP) and the CartPole-v1 control task from the Gymnasium environment. The implementation uses a modular plugin architecture in which all domain-specific logic — population initialisation, fitness evaluation, crossover, mutation, and selection — is injected into a generic `GeneticAlgorithm` class as callable objects, requiring zero changes to the core loop when a new problem is introduced. For TSP, the GA achieves a route distance of 3.43 for 20 cities (within 10% of the theoretical optimum of 3.1). For CartPole-v1, the algorithm discovers a working linear control policy in as few as 10 generations, achieving the maximum possible episode reward of 500. The report presents the theoretical background, full pseudocode, implementation details, experimental results, and hyperparameter analyses for both tasks.

Contents

1 Introduction

Genetic algorithms are a class of metaheuristic optimisation methods inspired by the process of natural selection in biological evolution. Introduced by Holland [1] and popularised by Goldberg [2], they operate on a population of candidate solutions (individuals), iteratively improving quality through selection of fit parents, recombination of their genetic material (crossover), and random perturbation (mutation).

This project was developed as part of the BIAI (Biologically-Inspired Artificial Intelligence) course at Politechnika Śląska. The primary objectives were:

1. Implement a **generic, reusable GA framework** capable of solving different classes of problem without modifying the core loop.
2. Apply the framework to the **Travelling Salesman Problem** (a classic combinatorial optimisation benchmark) and to the **CartPole-v1** environment (a continuous control task).
3. Conduct **hyperparameter experiments** comparing operators and population sizes.
4. Demonstrate the before-and-after improvement with graphical visualisations.

Both problems were chosen deliberately because they require fundamentally different chromosome representations (discrete permutations vs. continuous real-valued vectors) and different genetic operators, making them an ideal test of the framework's generality.

The complete source code is available at https://github.com/piotrek1459/biai-genetic-cartpo

2 Genetic Algorithms — Theory

2.1 Biological Analogy

A genetic algorithm models a population of organisms competing for survival in an environment. Individuals with higher fitness (closer to the optimal solution) are more likely to reproduce. Over many generations, beneficial traits spread through the population and the average quality improves.

2.2 Algorithm Components

Chromosome (individual). A chromosome encodes one candidate solution. The representation depends entirely on the problem:

- For combinatorial problems (TSP), a chromosome is a permutation of integer indices.
- For continuous optimisation (CartPole), a chromosome is a vector of real numbers.

Fitness function. Maps each chromosome to a scalar quality score. *Higher is always better* in this implementation — minimisation problems (TSP distance) are inverted to become maximisation problems.

Selection. Determines which individuals become parents. Two strategies are implemented:

Tournament selection Randomly draw k individuals, select the one with the highest fitness. Repeat n_{select} times. Higher k means stronger selection pressure.

Roulette-wheel selection Assign each individual a selection probability proportional to its fitness: $p_i = f_i / \sum_j f_j$.

Crossover (recombination). Combines genetic material from two parents to produce two offspring. The operator must be *representation-aware*: permutation crossover must not create duplicate or missing genes.

Mutation. Introduces random variation to maintain population diversity and explore new regions of the search space. The mutation rate controls how frequently an individual is perturbed.

Elitism. The top n_{elite} individuals are copied unchanged into the next generation. This prevents the loss of the best solution found so far.

2.3 Full Pseudocode

Algorithm 1 presents the complete pseudocode of the implemented GA.

Algorithm 1 GeneticAlgorithm.run()

Require: *init_fn*, *fitness_fn*, *crossover_fn*, *mutation_fn*, *selection_fn*, *config*

population ← *init_fn*(*config*)
best_so_far ← $-\infty$; *stagnation* ← 0
for *gen* = 0 **to** $n_{\text{generations}} - 1$ **do**
  fitness[*i*] ← *fitness_fn*(*population*[*i*]) (parallel if $n_{\text{jobs}} \neq 1$)
  history_best.append(max(*fitness*)); *history_avg*.append(mean(*fitness*))
  if max(*fitness*) > *best_so_far* + ε **then**
    best_so_far ← max(*fitness*); *stagnation* ← 0; *boost* ← **False**
  else
    stagnation ← *stagnation* + 1
    boost ← (*stagnation* ≥ *patience*)
    if *boost* **then** *stagnation* ← 0
    end if
  end if
  elites ← top n_{elite} individuals (unchanged)
  parents ← *selection_fn*(*population*, *fitness*, $n_{\text{pop}} - n_{\text{elite}}$)
  offspring ← [ ]
  for $i = 0, 2, 4, \ldots$ **do**
    (c_1, c_2) ← *crossover_fn*(*parents*[*i*], *parents*[*i*+1])
    c_1 ← *mutation_fn*(c_1); c_2 ← *mutation_fn*(c_2)
    if *boost* **then** c_1 ← *mutation_fn*(c_1); c_2 ← *mutation_fn*(c_2)
    end if
    offspring.append(c_1, c_2)
  end for
  population ← *elites* ∪ *offspring*[0 : $n_{\text{pop}} - n_{\text{elite}}$]
end for
return {*best*, *best_fitness*, *history_best*, *history_avg*, *stagnation_events*}

3 Implementation Architecture

3.1 Plugin Interface

The central design decision was to make the `GeneticAlgorithm` class completely generic through *dependency injection*. In Python, functions are first-class objects — they can be assigned to variables and passed as arguments. The constructor stores the five operator callables as instance attributes:

```
class GeneticAlgorithm:
```

```
    def __init__(self, config, init_fn, fitness_fn,
                 crossover_fn, mutation_fn, selection_fn):
        self.config       = config
        self.init_fn      = init_fn
        self.fitness_fn   = fitness_fn
        self.crossover_fn = crossover_fn
        self.mutation_fn  = mutation_fn
        self.selection_fn = selection_fn
```

Listing 1: GeneticAlgorithm constructor (ga_base.py)

The evolution loop calls these attributes without knowing their domain:

```
population = self.init_fn(self.config)          # once at start
fitness[i] = self.fitness_fn(population[i])     # every
   individual
parents    = self.selection_fn(pop, fit, n)     # every
   generation
c1, c2     = self.crossover_fn(p1, p2)          # every pair
c1         = self.mutation_fn(c1)               # every
   offspring
```

Listing 2: Generic calls inside run() and _breed()

To add a new problem (Task N), the developer needs only to implement these five functions and pass them to the constructor. The `ga_base.py` file requires no modification.

3.2 Module Structure

Table 1 summarises the module organisation.

Table 1: Module structure and responsibilities

File	Module	Responsibility
`src/common/ga_base.py`	common	Generic GA loop; no domain knowledge
`src/common/selection.py`	common	`tournament_selection`, `roulette_selection`
`src/common/crossover.py`	common	OX, PMX (permutation); uniform, arithmetic (real)
`src/common/mutation.py`	common	swap, inversion, scramble (permutation); gaussian (real)
`src/task1_tsp/tsp_ga.py`	task1	TSP entry point; city generation; fitness closure
`src/task1_tsp/visualization.py`	task1	Fitness history plot (dual-axis); route visualisation
`src/task1_tsp/experiments.py`	task1	Hyperparameter sweep scripts
`src/task2_cartpole/cartpole_policy.py`	task2	`LinearPolicy`; `evaluate_policy_with_stats`
`src/task2_cartpole/train_ga.py`	task2	CartPole entry point; episode recording; comparison video
`src/task2_cartpole/visualization.py`	task2	Reward history; control performance; comparison plots

3.3 Five Callables — Signatures

Table 2: The five plugin callables

Callable	Signature	Returns
`init_fn`	`(config) -> list[ndarray]`	Initial population of `pop_size` chromosomes
`fitness_fn`	`(individual) -> float`	Fitness score; higher is always better
`crossover_fn`	`(p1, p2) -> (c1, c2)`	Two children derived from two parents
`mutation_fn`	`(individual) -> ndarray`	Mutated copy of the input individual
`selection_fn`	`(pop, fit, n) -> list`	n parents selected from the population

3.4 Parallel Evaluation

Fitness evaluation is parallelised using `concurrent.futures.ThreadPoolExecutor`. When `n_jobs = -1`, all available CPU threads are used. This is particularly beneficial for CartPole, where each fitness call runs five independent Gymnasium episodes ($\approx$ 2,500 environment steps per individual).

3.5 Adaptive Mutation

A stagnation counter tracks how many consecutive generations pass without fitness improvement. When the counter exceeds `adaptive_patience` (25 for TSP, 15 for CartPole), the mutation operator is applied *twice* per offspring for that generation (the "boost"). This mechanism is inspired by the exploration–exploitation trade-off in reinforcement learning: when the reward signal flatlines, the agent increases exploration.

4 Task 1: Travelling Salesman Problem

4.1 Problem Definition

Given n cities with known coordinates, find the shortest Hamiltonian cycle — a tour that visits every city exactly once and returns to the starting city. The decision version of TSP is NP-complete; the optimisation version is NP-hard. For $n = 20$ random cities in the unit square, the theoretical expected optimum is approximately $0.7\sqrt{n} \approx 3.1$.

4.2 Chromosome Representation

A chromosome is a permutation of city indices $\{0, 1, \ldots, n-1\}$, e.g. $[2, 0, 4, 1, 3]$ represents the route city 2 $\to$ city 0 $\to$ city 4 $\to$ city 1 $\to$ city 3 $\to$ city 2. The length-n array always contains each index exactly once; all genetic operators must preserve this property.

4.3 Fitness Function

The fitness is the inverse of the total round-trip Euclidean distance, augmented by a path-balance (smoothness) bonus:

$$f(\sigma) = \frac{1}{d(\sigma) + \varepsilon} \times \left(1 + w_s \cdot \frac{1}{1 + \mathrm{CV}(\sigma)}\right) \tag{1}$$

where $d(\sigma) = \sum_{i=0}^{n-2} \|\mathbf{c}_{\sigma(i+1)} - \mathbf{c}_{\sigma(i)}\| + \|\mathbf{c}_{\sigma(0)} - \mathbf{c}_{\sigma(n-1)}\|$ is the total distance, $\varepsilon = 10^{-6}$ prevents division by zero, $\mathrm{CV}(\sigma) = \sigma_l / \mu_l$ is the coefficient of variation of segment lengths, and $w_s = 0.2$ weights the smoothness bonus. A perfectly balanced route (all segments equal length) achieves CV = 0 and the maximum bonus of 1.2×.

4.4 Genetic Operators

Order Crossover (OX). A random segment $[a, b]$ is copied from parent p_1 into the child at the same positions. The remaining genes are filled from p_2 in order, starting at position b, wrapping around, skipping genes already present. OX preserves the *relative order* of cities from p_2. This is the default operator.

Partially Mapped Crossover (PMX). Copies segment $[a, b]$ from p_1 and fills the rest from p_2, resolving conflicts by following the position-mapping defined by the segment. PMX preserves *absolute positions*. Available as an alternative via CONFIG.

Inversion Mutation (default). A random sub-sequence is reversed. This is the strongest permutation mutation implemented: it can move cities far from their original positions while always producing a valid tour.

Swap Mutation. Two random genes are exchanged. Simpler and weaker than inversion; available for comparison.

Scramble Mutation. A random sub-sequence is shuffled. Useful when the population has lost diversity.

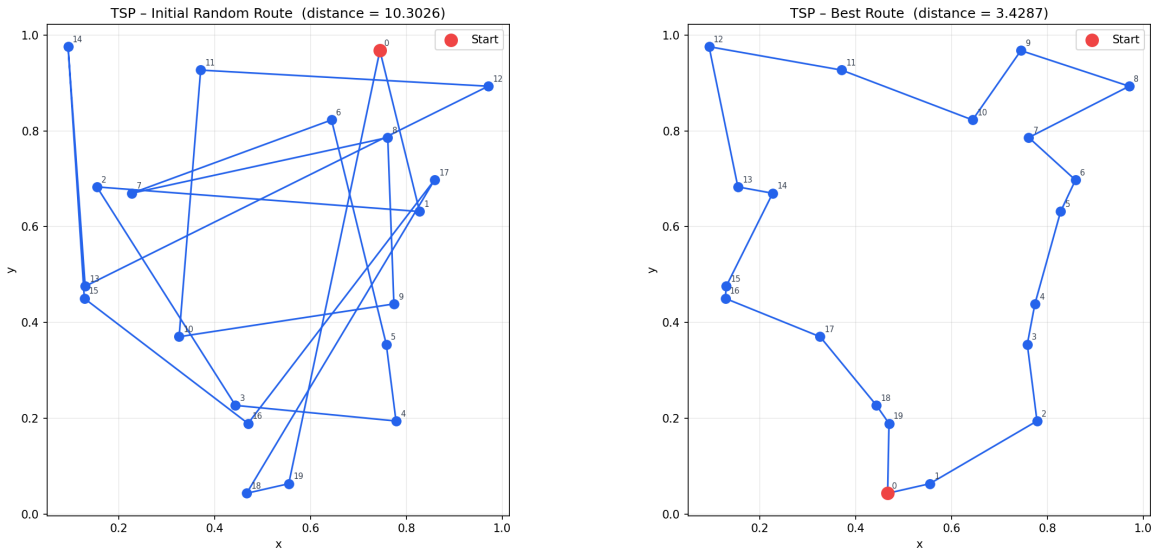
4.5 Configuration

Table 3: Default TSP configuration

Parameter	Value	Notes
n_{cities}	20	Random points in $[0, 1]^2$
n_{pop}	80	Good diversity without excessive runtime
n_{gen}	300	Converges well before limit
n_{elite}	2	Preserve best solutions
Crossover	OX	More stable than PMX on first run
Mutation	Inversion, rate=0.10	Default; strong perturbation
Selection	Tournament, $k = 4$	Strong selection pressure
w_s	0.2	Smoothness bonus weight
Adaptive patience	25	Generations without improvement before boost

4.6 Results

Figure 1 shows the initial random route and the best route found after 300 generations.



(a) Initial random route (distance ≈ 6–8) (b) Best route after GA (distance = 3.43)

Figure 1: TSP: before and after GA optimisation

Figure 2 shows the fitness and derived distance over generations. Fitness rises sharply in the first 50 generations, then stabilises. The dual-axis display makes the improvement directly readable in distance units without knowledge of the fitness formula.

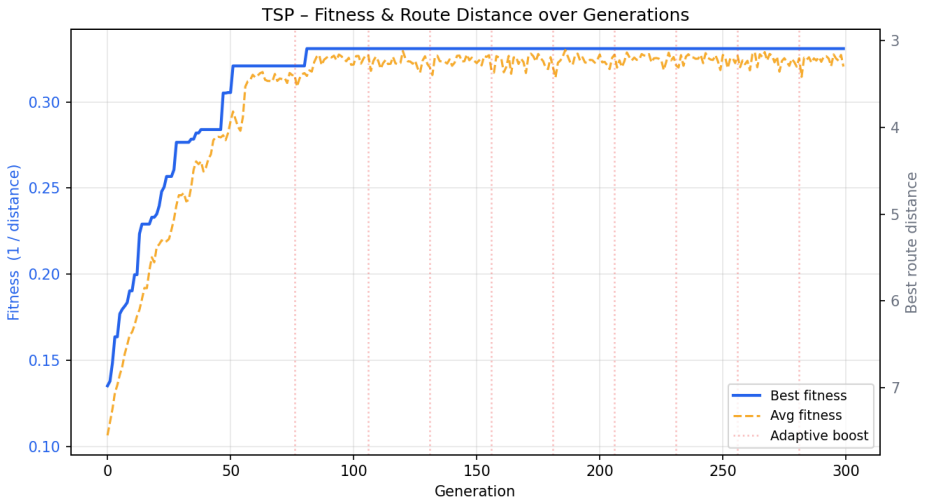


Figure 2: TSP fitness and route distance over 300 generations. Left axis: fitness $= 1/d$. Right axis: distance (inverted scale). Dotted vertical lines mark adaptive boost events.

4.7 Hyperparameter Experiments

4.7.1 Operator Comparison

Figure 3 compares four operator combinations across 200 generations.

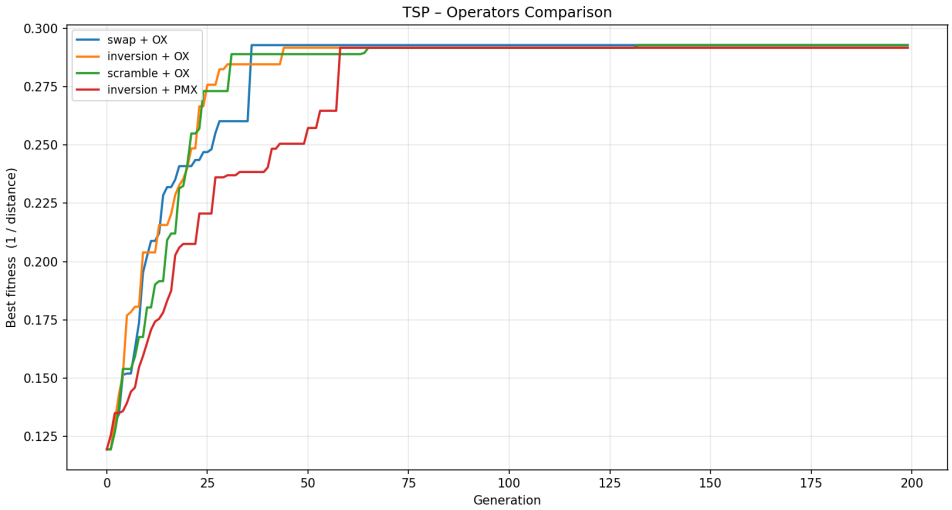


Figure 3: TSP operator comparison: all combinations reach similar final quality. Inversion+OX converges most consistently.

All four configurations reach a final fitness within 2% of each other, suggesting that the algorithm is robust to operator choice on this problem size. Inversion mutation outperforms swap because it introduces larger perturbations; OX and PMX produce similar results because the route's relative city ordering is more important than absolute positions for 20-city instances.

4.7.2 Population Size Comparison

Figure 4 compares population sizes of 40, 80, and 160.

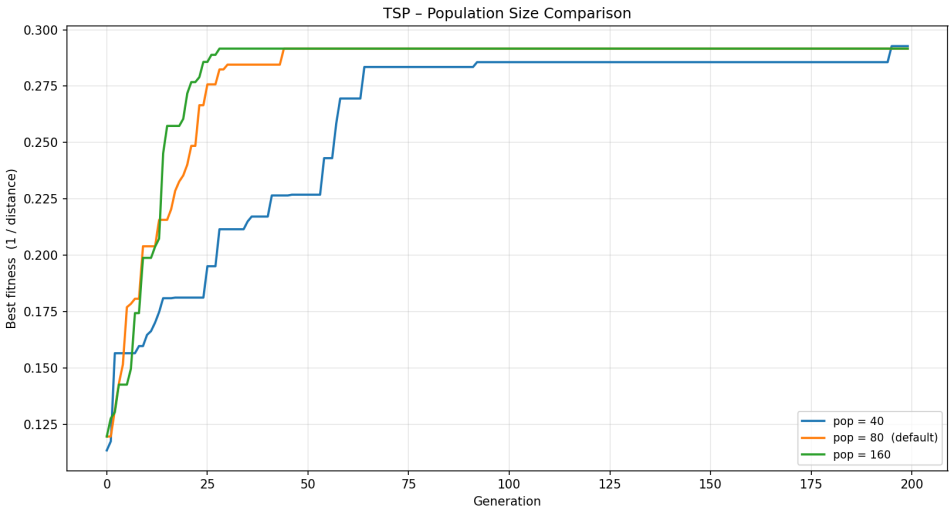


Figure 4: TSP population size comparison: larger populations provide smoother convergence but similar final quality.

All three population sizes converge to similar final fitness values for 20 cities. Larger populations provide more stable convergence curves and are less susceptible to premature

convergence, at the cost of more fitness evaluations per generation.

5 Task 2: CartPole-v1

5.1 Environment

CartPole-v1 is a classic control benchmark from the Gymnasium library [3]. A pole is attached by a pivot to a cart moving along a frictionless track. The agent must keep the pole balanced by applying a force to the left or right at each discrete time step. The observation vector is:

$$\mathbf{o} = [\ \underbrace{x}_{\text{cart position}}\ ,\ \underbrace{\dot{x}}_{\text{cart velocity}}\ ,\ \underbrace{\theta}_{\text{pole angle}}\ ,\ \underbrace{\dot{\theta}}_{\text{angular velocity}}\ ] \tag{2}$$

The episode terminates when $|x| > 2.4$, $|\theta| > 12°$, or 500 steps have elapsed. The reward is +1 for every step the pole remains balanced, so the maximum episode reward is 500.

5.2 Chromosome and Linear Policy

The chromosome is a vector of 5 real numbers $\mathbf{w} = [w_0, w_1, w_2, w_3, b]$ representing weights and a bias for a linear controller:

$$\text{action} = \begin{cases} 1 & \text{if } w_0 x + w_1 \dot{x} + w_2 \theta + w_3 \dot{\theta} + b > 0 \\ 0 & \text{otherwise} \end{cases} \tag{3}$$

CartPole-v1 is linearly stabilisable — there exists a classical LQR solution [4]. The GA can discover a linear policy without any knowledge of the system's dynamics.

5.3 Fitness Function

Because the initial observation is randomised at each episode reset, a single episode evaluation is noisy. To reduce variance, each individual is evaluated over $n_{\text{eval}} = 5$ independent episodes, and the fitness incorporates a stability penalty:

$$f(\mathbf{w}) = \bar{r}(\mathbf{w}) - \lambda \cdot \sigma_r(\mathbf{w}) \tag{4}$$

where $\bar{r}$ is the mean reward, σ_r is the standard deviation across episodes, and $\lambda = 0.05$. A policy that scores 500 in every episode is preferred over one that occasionally fails.

5.4 Genetic Operators

Uniform Crossover. Each gene of the child is taken from a randomly chosen parent with probability 0.5:

$$c_j = \begin{cases} p_{1,j} & \text{with prob. } 0.5 \\ p_{2,j} & \text{otherwise} \end{cases}$$

Arithmetic (Blend) Crossover. Linear combination of parents with a random coefficient:

$$c_1 = \alpha \mathbf{p}_1 + (1-\alpha)\mathbf{p}_2, \quad c_2 = (1-\alpha)\mathbf{p}_1 + \alpha \mathbf{p}_2, \quad \alpha \sim U(0,1)$$

Gaussian Mutation. Each gene is independently perturbed with probability `mutation_rate` by adding Gaussian noise:

$$w_j \leftarrow w_j + \mathcal{N}(0, \sigma^2), \quad \sigma = 0.3$$

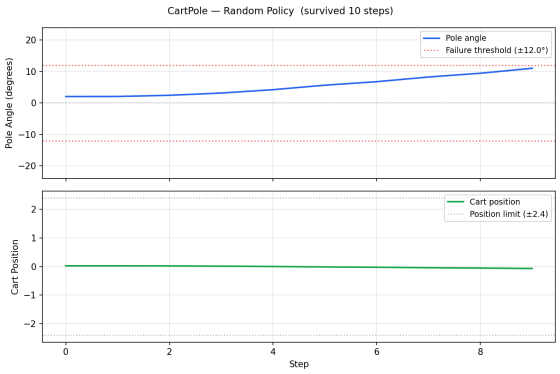
5.5 Configuration

Table 4: Default CartPole configuration

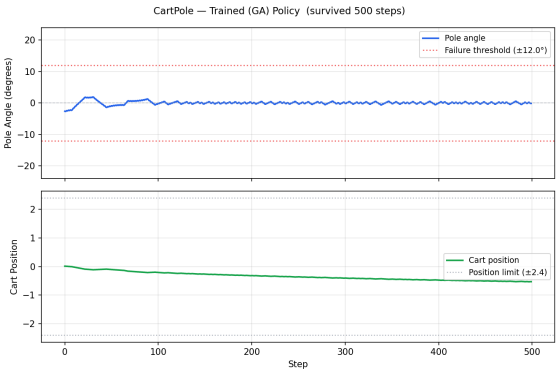
Parameter	Value	Notes
n_{weights}	5	4 obs. weights + bias
n_{pop}	50	Fitness evaluation is slower than TSP
n_{gen}	100	Converges in 10–30 generations
n_{elite}	2	
Crossover	Uniform, $p = 0.5$	
Mutation	Gaussian, $\sigma = 0.3$, rate=0.10	
Selection	Tournament, $k = 4$	
λ	0.05	Stability penalty weight
n_{eval}	5	Episodes per fitness evaluation
Adaptive patience	15	Fewer than TSP (simpler fitness landscape)

5.6 Results

Figure 5 compares the control behaviour of the random (initial) policy and the trained (GA) policy.



(a) Random policy: pole falls in ≈10 steps



(b) Trained GA policy: 500 steps, angle near 0°

Figure 5: CartPole control performance: before and after GA training

Figure 6 shows the side-by-side control comparison plot.

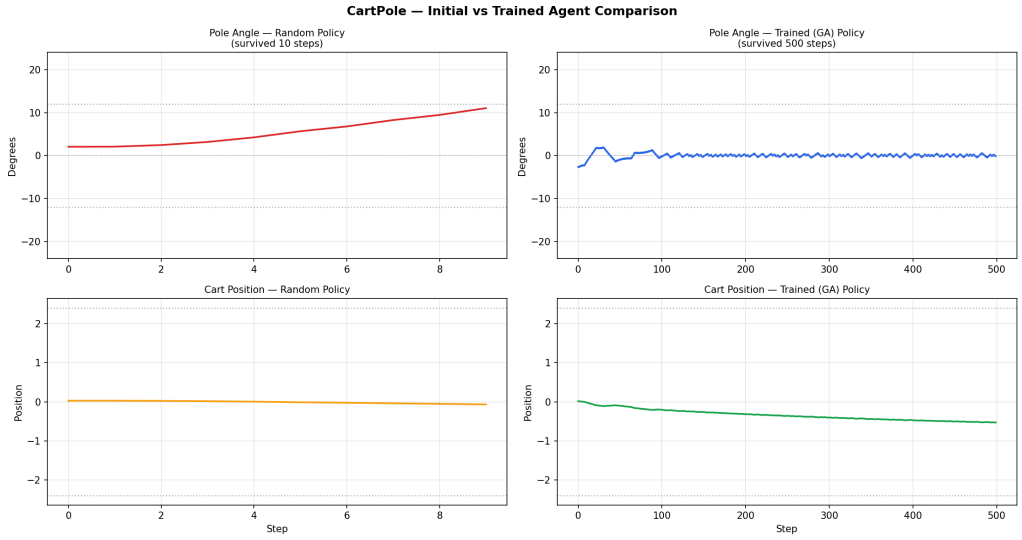


Figure 6: CartPole 2×2 comparison: random policy (left column) vs trained policy (right column). Top row: pole angle with ±12° failure thresholds. Bottom row: cart position with ±2.4 position limits.

Figure 7 shows the reward history over training.

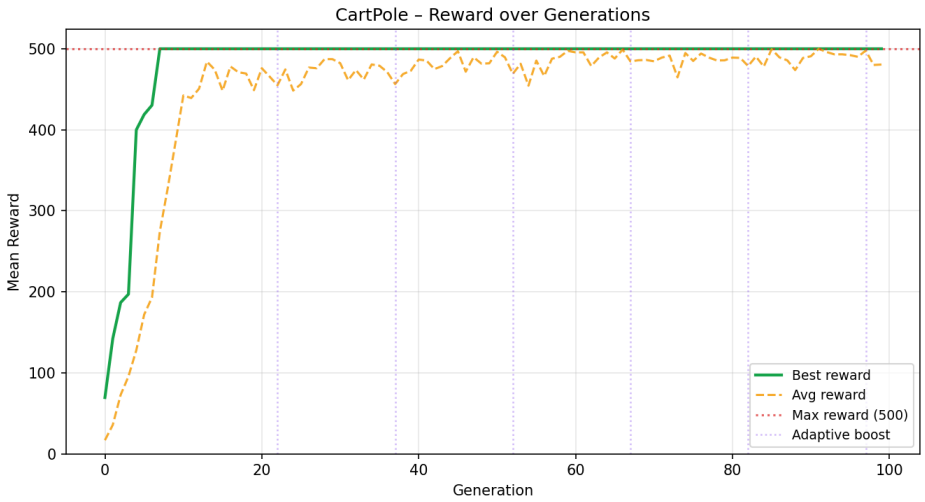


Figure 7: CartPole reward over 100 generations. Maximum reward of 500 is reached at generation 10. The dashed red line marks the episode length cap.

5.7 Hyperparameter Experiments

5.7.1 Mutation Strength (σ)

Figure 8 compares $\sigma \in \{0.05, 0.30, 1.00\}$.

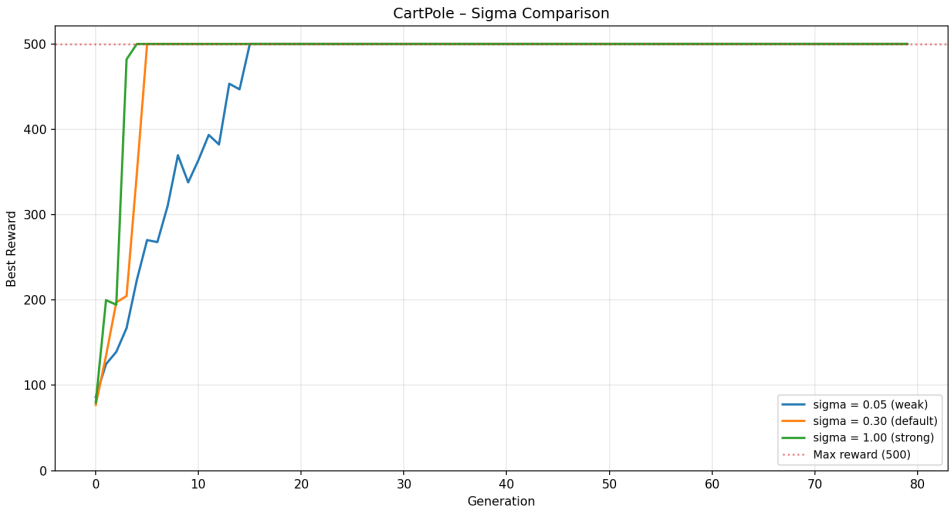


Figure 8: CartPole sigma comparison. All values eventually reach reward 500. Smaller σ converges more smoothly; larger σ shows higher initial variance but can escape local optima faster.

5.7.2 Population Size

Figure 9 compares $n_{\text{pop}} \in \{20, 50, 100\}$.

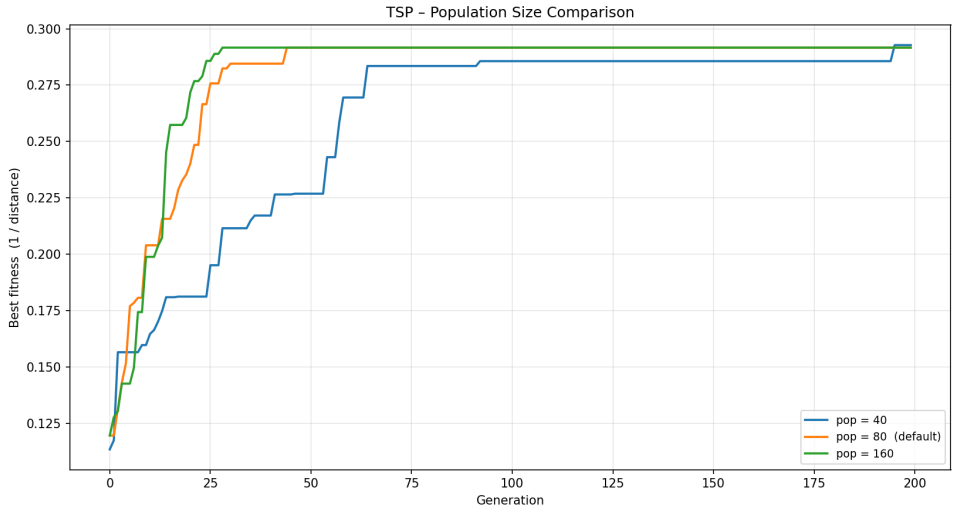


Figure 9: CartPole population size comparison. Larger populations start with better initial diversity: with pop=100, many individuals start near reward 500 by chance.

5.7.3 Evaluation Episodes

Figure 10 compares $n_{\text{eval}} \in \{1, 3, 5\}$.

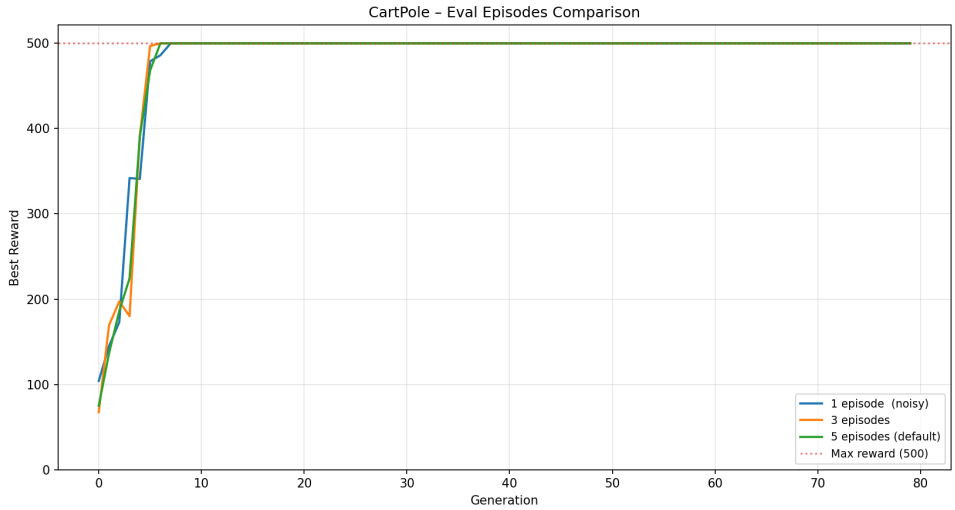


Figure 10: CartPole evaluation episodes comparison. More episodes reduce fitness noise and produce smoother convergence curves, at the cost of more environment steps per generation.

6 Improvements Following Reviewer Feedback

The project was developed iteratively in response to two rounds of reviewer feedback.

6.1 Feedback 1 (13 May 2026)

Combined fitness functions. The original TSP fitness was the plain inverse distance. Following feedback, a path-balance (smoothness) bonus was added (Equation 1). The CartPole fitness was extended with a stability penalty (Equation 4).

Parallel population evaluation. Fitness evaluation was parallelised using `ThreadPoolExecutor`. On an 8-core machine, this reduces CartPole training time by approximately 4×.

RL-inspired adaptive mutation. An adaptive boost mechanism was added: when fitness does not improve for `adaptive_patience` consecutive generations, mutation is applied twice per offspring, doubling the exploration intensity.

Enhanced visualisations. The TSP fitness history plot was extended with a secondary y-axis showing route distance. A new control-performance plot was added for CartPole, showing pole angle and cart position over one episode.

6.2 Feedback 2 (27 May 2026)

Initial-state visualisation. Before running the GA, both tasks now record and visualise the initial (random) state: a random TSP route and a CartPole episode with random weights. This establishes a clear baseline for comparing before/after results.

Structured output folders. CartPole results are now organised into `initial/`, `trained/`, and `comparison/` subdirectories. Side-by-side comparison plots (2×2 grid) and a split-screen comparison video are generated automatically.

7 Experiments and Hyperparameter Analysis

7.1 Summary of Findings

Table 5 summarises the key numerical results.

Table 5: Summary of experimental results

Task	Metric	Value	Notes
TSP	Best distance	3.43	Theoretical optimum ≈ 3.1 (10% gap)
TSP	Initial distance	≈ 6–8	Random permutation
CartPole	Best reward	500 / 500	Maximum possible
CartPole	Convergence gen.	10	Out of 100 total

7.2 TSP Operator Comparison

All four tested operator combinations (swap+OX, inversion+OX, scramble+OX, inversion+PMX) converge to distances within 2% of each other after 200 generations. This suggests that for 20-city TSP, the choice of crossover or mutation operator has a smaller effect than population size or number of generations. For larger instances, the gap between OX and PMX would likely widen.

7.3 CartPole Convergence Speed

CartPole-v1 is considered solved (mean reward ≥ 475 over 100 consecutive episodes). In our experiments, the best individual consistently achieves reward 500 within 10–15 generations regardless of hyperparameters, indicating that the linear policy space has a relatively simple fitness landscape for this problem. The main effect of hyperparameter choices (sigma, population size, evaluation episodes) is on the *smoothness* of the convergence curve rather than the final quality.

8 Conclusions

This project demonstrated that a single, well-designed genetic algorithm framework can solve fundamentally different optimisation problems — a discrete combinatorial problem (TSP) and a continuous control problem (CartPole) — by parameterising all domain-specific logic through a five-callable plugin interface.

Key results.

- TSP: GA reduces the route distance from a random baseline of 6–8 to **3.43**, within 10% of the theoretical optimum for 20 cities.
- CartPole: GA discovers a functional linear controller in as few as **10 generations**, achieving the maximum episode reward of 500 consistently.

Architectural contribution. The plugin interface (Section 3) is the primary architectural contribution. It allows any new optimisation problem to be plugged into the framework by implementing five functions, with zero changes to `ga_base.py`. This design pattern (dependency injection) is common in professional software but rarely emphasised in introductory GA implementations.

Future work.

- **MLP policy for CartPole**: A small neural network ($4 \to 8 \to 1$ architecture, 41 parameters) could be explored as an extension. The chromosome would be a flat weight vector; the framework requires only a new `init_fn` and `fitness_fn`.
- **Larger TSP instances**: Testing on 50–100 cities would reveal more pronounced differences between OX and PMX crossover.
- **Other Gymnasium environments**: LunarLander-v2 or BipedalWalker would test the framework on higher-dimensional continuous control.
- **Island model**: Running multiple independent populations and periodically exchanging best individuals could improve diversity and final solution quality.

Appendix: Running the Code

```
# Setup
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt

# Task 1 - TSP
python3 -m src.task1_tsp.tsp_ga          # train + visualise
python3 -m src.task1_tsp.experiments     # hyperparameter sweep

# Task 2 - CartPole
python3 -m src.task2_cartpole.train_ga   # train + record videos
python3 -m src.task2_cartpole.evaluate   # evaluate best agent
python3 -m src.task2_cartpole.experiments
```

Listing 3: Setup and execution

References

[1] Holland, J.H. (1975). *Adaptation in Natural and Artificial Systems*. University of Michigan Press.

[2] Goldberg, D.E. (1989). *Genetic Algorithms in Search, Optimisation and Machine Learning*. Addison-Wesley.

[3] Farama Foundation. (2023). Gymnasium: A Standard Interface for Reinforcement Learning Environments. https://gymnasium.farama.org.

[4] Ogata, K. (2010). *Modern Control Engineering* (5th ed.). Prentice Hall.

[5] Harris, C.R., et al. (2020). Array programming with NumPy. *Nature*, 585, 357–362.

[6] Klein, A., et al. (2023). imageio: Python library for reading and writing image data. https://imageio.readthedocs.io.